Chapter 3: Meaning Fields and the Hermeneutic Calculator

Abstract

In this chapter I explore meaning in mathematics. I use meaning field analysis, a qualitative research method, to analyze students' work. This affords a justification of the anaphoric conceptualization of number that I posited in chapter 1. I then turn to the Hermeneutic Calculator. The Hermeneutic Calculator refers to an iPhone app I am developing that does math like students – erroneously. It also refers to a grander set of all possible mathematical doings that could be algorithmized.

Introduction

In the previous chapter, I addressed the critical, justificatory, and ampliative moments of the concept, and discussed how meaning depends on use in analytic pragmatism. In this chapter, I explore meaning more generally. I address how to understand multiple meanings, including disconnected justifications from the previous chapter, through the notion of *meaning fields*. I also discuss algorithmic elaboration with finer granularity by explicating the code of the *Hermeneutic Calculator*. To do this, I discuss some of the rules for how to add algorithms to the *Hermeneutic Calculator*. The *hermeneutic calculator* is named so along the typology of knowledge constitutive interests (Habermas, 1971). In that typology, each 'higher' level includes the 'lower' levels. Empirical/analytic knowledge is the first type and it relates to making good predictions. Casio calculators or TI-84s would be more or less worthless if they did not give predictable answers. We would simply throw such a machine away. The hermeneutic knowledge constitutive interest is related to reaching understanding. The calculator that I imagine programming is not unbound from rules, so correctness/incorrectness is still an issue – but the *hermeneutic calculator* does mathematics incorrectly some of the time. It purports to compute as learners do and so is rules-based but the rules are generated from empirical examples of student work. For completeness in the typology, the last knowledge constitutive interest is the critical/emancipatory interest which depends on first-person reflection.

I develop the notion of *projective validity* as a kind of pragmatist recipe for how to analyze a student work sample and then construct a *projected inference* that is a kind of prediction. My basic idea for projective validity is that the rules (i.e. norms) governing the subsentential elements of speech (singular terms and predicates) are repeatable. If we can discern

the rules that govern those constituent parts, we can then use those rules in different contexts. Following such rules does not assure the acceptability of the new speech act, but there is a particular class of normative validity claims that relate to the material inferential rules for the parts of speech.

It is interesting to contemplate what makes those rules repeatable. I think it has to do with the deontic aspect of the deontic-normative modality. Deontology is about the hold our concepts have on us, which is our responsibility to the concepts we use. In folk psychology, projection is a species of taking the attitude of the other while importing one's own not-necessarily-extendible commitments, validity claims, or desires. The rulishiness of concepts is extended out past their original material contexts in such projections. Extendibility is an important feature of whatever it means to be a rule. Consequently, I think it makes sense to talk about projection as being enabled through a larger commitment to normativity. Without holding ourselves responsible to the rulishiness of concepts, we could not appropriately extend those concepts past their initial uses. This commitment to normativity is the *in*finite {I}. Being a normative subject entails commitment to the repeatability (if not the genuine repeatability discussed in the previous chapter or in chapter 4) of conceptual contents. So with projection, I posit that the {I} is more strongly located within the act of projection (i.e. the constituent substitution inferences), than it is with the speech act. The projective speech act just follows the finite rules of concepts. In linear algebra, orthogonal projection involves reducing a vector's dimensionality, transforming a vector like (x, y, z) to $(x, y, 0)$. Imagistically, we only see the shadow of the 3d arrow in 2d space. So, imagistically, with projection the {I} is not risked within the content of the projected speech act – the {I} is the 'missing dimension.'

When I took standardized tests in grade school there were often questions with a bunch of nonsense words in them. I don't remember what they were testing, but they said things like "a *lep* is a hammer, a *tay* is a block of wood." With those simple material substitution inferences in place one can make sense of a sentence like "I used the *lep* to nail the *tay* to the wall," at least providing that the sentence "I used the hammer to nail the block of wood to the wall" is within the horizons of intelligibility for whomever is trying to interpret the sentence. This is a kind of projection, though not quite as rigid as *inferential projection* as Brandom lays the concept out. The "validity" here has to do with whether or not those rules are extendable. Suppose *A* utters the nonsense sentence and *B* says "What?!" *A* could then say "Oh, that sentence is okay because a *lep* is a hammer and a *tay* is a block of wood. Didn't you take the iStep test in 4th grade?" This is a species of normative validity because it does not reference the objective nature of the claim. In no way would it be necessary to hire an investigative reporter to determine whether I had, in fact, hammered the wood to the wall. Nor is the claim subjective. It does not reference the privileged realm of mind, unlike the sentence "I felt terrible when I hammered the wood to the wall." While it is rather hard to argue the point, I think the {I} is located and staked on the substitution inferences like *lep* for hammer etc. The {I} is staked on the projection more so than the totality of the speech act. Our concepts pull us out away from our embodied experience with the deontic rope, the movement of Spirit, being the tugging apparatus.

Projective validity is normative, it has to do with meaning and does not reference any reality external to a linguistic reality. Subjective validity claims reference a domain with privileged access (i.e. *my* thoughts). Objective validity claims reference a non-linguistic shared domain (i.e. "that rock is igneous" where the rock in question is not internal to the act of referencing). Originally, I thought projective validity was a species of objective validity – in part

because it is based on making predictions, but these are predictions about meaning and meaning is, generally, normative. Moreover, when I code student work samples below, there is not some *thing* out there which might resist my claims. Claiming a rock to be squishy, for example, would be met with a hard resistance. In this work, it would be a person who resists and say "no, that is not what I (or they) meant at all." The resistance to the claim is not external to the claim, instead it references the claim itself.

Before I get too far into describing *projective validity*, it is important to state clearly that, at this point, my thinking is very constrained. Inferential projection is somewhat opaque in Brandom's work, and in this rendering my limited understanding produces a limited application. Future work on the topic needs to address the dialectic between necessity and possibility under the idea that such a dialectic relates to time itself. Inferential projection somehow plucks out a sentence from a vast manifold of possible speech acts but I do not have the details worked out yet. So I will stick with the constrained version for now.

The basic thought behind the *Hermeneutic Calculator* is that incompatibility entailment works by taking as wide a field of meanings as can be thought, determining what is incompatible with what, and then determining entailments. Recall from chapter one that we say that A incompatibility entails B if everything incompatible with B is also incompatible with A. I use both incorrect and correct student samples to generate as wide a field as possible, so the *Hermeneutic Calculator* generates incorrect mathematics. It ought not be used for doing one's taxes. The *Hermeneutic Calculator* would then take *information* to rule out some of these possibilities: "A signal is informative, contains or expresses information, just insofar as it rules out some alternatives" (Brandom, 2019, p. 103). This definition of *information* is intimately

related to time. Time rules out some possibilities. This work is limited by the absence of a theory of time.

The data I work with here is deliberately quite simple compared to the kinds of models of student thinking that abound in mathematics education research. The next stage in the development of this calculator would be to algorithmize those more interesting models to create models of students as determinate pathways through the *Hermeneutic Calculator*. Backgrounded throughout this is my naïve opinion that artificial intelligence will not feel human until it is 'bad' at math in specifically human ways. That is, no AI will pass my personal Turing test until it can justify why $\frac{2}{3} > \frac{3}{4}$.

In this chapter, I populate part of that reality with student work samples. Is it possible that $\frac{2}{3} > \frac{3}{4}$? It is under the rule "the smaller the denominator, the larger the fraction," which holds when numerators are held constant. We could name this the world in which fractions are ordered in this manner a *possible world* and, in earlier drafts of this document I did just that. However, it's a bit misleading to do this because it is not some far-off realm where fractions are ordered thusly. It's our present reality. Students make this error somewhat routinely. "The picture (of Hegel's inverted world) is one according to which each actual state of affairs, each fact, is surrounded by a penumbra of merely possible, strongly contrasting states of affairs" (Brandom, 2019, p. 140). So instead of referring to possible worlds below, I am instead going to refer to the manifold of possible speech within a particular horizon of intelligibility, or the *manifold* for short. The penumbra of errorful algorithms surrounds our correct conceptual deployments but that penumbra is still within the manifold of possible speech with the important caveat that some recognitive communities will count those algorithms as errors and so the utterers of those errors

will not have their synthetic needs met through those erroroneous speech acts. This may seem like a throw-away line, but its actually important for the work of teaching. Instead of saying "you are wrong," the critical expressivist mathematics educator might say "others may not understand what you are trying to say. I get it, and its good thinking, but think a little bit more about how well it fits into the larger system." I am practically directly quoting Phil Carspecken's responses to some of my erroneous work in theory. The recognition desire impels us to work to be understood, and so clarifying where those breakdowns might be happening is not a dictatorial move – education is, at least partially, an enabling condition for recognition.

In its grandest formulation, the *Hermeneutic Calculator* is constituted in this manifold, addressing all possible mathematical doings that are within some horizon of intelligibility. What I actually accomplish here barely hints at that ambition. On the formal side, the work that mathematicians do on sets of axioms, including the work in automated theorem proving, form the other side of the Hermeneutic Calculator. The sense of formal mathematics may depend on thought, but it need not reference a thinker. The side that I am trying to fill out is both sense and reference dependent on thought. There is a path from the broader notion of mathematics I present here to the more specific kind of mathematical activity of professional thinkers. Formal inferences can be derived from material inferences according to the recipe that Brandom discusses.

> One important consideration is that the notion of formally valid inferences is definable in a natural way from that of materially correct ones, while there is no converse route. For given a subset of vocabulary that is privileged or distinguished somehow, an inference can be treated as good in virtue of its form, with respect to that vocabulary, just in case
>
> It is a materially good inference, and

> It cannot be turned into a materially bad one by substituting nonprivileged for nonprivileged vocabulary in its premises and conclusions" (Brandom, 2000, p. 11).

Teaching and learning allow for information to reduce these possibilities from the combinatorially intense state of *confusion* towards the promise of a 'well-ordered mind'. We learn by snuffing out possibilities with information.

Churning in the background here are some important distinctions between the alethic modal and deontic normative vocabularies. *Possibility* is a term in the alethic modal vocabulary. I take it that the deontic-normative vocabulary is a pragmatic metavocabulary for the alethic modal vocabulary so what constitutes "possibility" is normatively construed. I take a pretty loose interpretation of the term saying more or less that "This student said that A was so, so it is possible to think that A is so," rather than saying "To think A is not to think at all since A is an impossible thought." Rödl (2018) takes the latter approach but it strains the notion of thought to insist that only some thoughts are thoughts. Future work may find some profit in tidying the looseness of my notion of possibility up. Specifically, the relationship of modal realism to conceptual realism, and the integration of conceptual realism into Habermas' critical theory are areas that I have not thought through with enough clarity to relate. In general, my thoughts on metaphysical logic are incomplete.

The risk of invoking such a loose notion of possibility is devolution into an indeterminate soup of possibilia. To avoid this potentially sloppy mess, I repurpose Brandom's (2000) notion of *projective inferences*. Brandom uses this notion in a bid to understand how new assertions can be understood by competent speakers. *Projective inferences* 'stay in their lane.' My usage leaves the notion of *projective inference* free to do the work of enacting boundaries between what is

compossible. A field of assertions and explicit inferences between them form the initial conditions of a separable compossible region in the manifold while the set of *projective inferences* that are enabled by those initial conditions, which are normatively regulated, flesh out the boundaries of those compossible regions. The discussion within this paragraph is a high-level view for what shall become the minutae of *projective validity* that I discuss below.

In terms of organization, I shall first discuss *meaning fields* and how they enable a conversation about *hybridized models of quantity*. This allows for a non-formal definition of "and" and "or," based on determinatively carving up of the soup of possibilia into individual compossible regions in the manifold. Next, I will discuss *projective validity* which enables a conversation about how to embody the Hermeneutic Calculator. What I present as the Hermeneutic Calculator is just a few algorithms, written in SWIFT. I am not much of a programmer, so these algorithms are very unsophisticated, but I think they gesture in the correct direction that someone more talented might pursue so I include them.

Meaning Fields

There are many qualitative research methods that are under-explored in mathematics education, presumably because mathematical knowledge is generally construed according to the empirical-analytic knowledge constitutive interest, but I have found *meaning field analysis* to be particularly generative in part because it gives combinatorial weight to the experience of confusion. We talk about information overload, but in the negative sense of information that I articulated in the previous section, information is the reduction of possibility, so the problem with confusion is not information overload but is rather a lack of information. When we experience confusion, we are experiencing an overwhelming sense of possibility.

In the following discussion, my aim is to relate *meaning fields* to the manifold of possible speech. *Meaning fields* were developed within a Habermasian critical research methodology (P. Carspecken, 1995). *Meaning field analysis* is a qualitative research method whereby researchers interpret a speech act as having multiple possible interpretations. "You cannot know for certain what an actor intended with her act, you cannot know for certain what impressions of meaning were received by those witnessing the act or directly addressed by the act, but you can specify possibilities" (P. Carspecken, 1995, p. 96). When performing meaning field analysis, the researcher obtains a list of plausible interpretations of a communicative act. The more ambiguous the communicative action is, the more populated with possible interpretations its representation through meaning field analysis will be. To be clear, the meaning field itself is not this analysis. Rather, the meaning field is the holistic experience of meaning in communicative contexts. The other speaks and we are not always certain of what they mean, but there is a host of possible interpretations with some foregrounded and some less plausible. When we take our turn in communicating with this other, we make inferences about what that meaning might be. But there is not one sole, singular meaning. The meaning of a speech act is the host of possible meanings.

It is probably worth mentioning that meaning fields are not the same thing as intended meanings. Intentionality is the rational reconstruction of an impetus to act. As a rational reconstruction, intention conforms that act to a normative order of reason. The goodness of intention is retrospective in nature and only comes about when either encountering (or rehearsing) the "no" of another person. Intentions, insofar as the actor can rationally construct the impetus to act, depend on those representations of rationality for their goodness.

Meaning field analysis is meant to extend and make explicit the assumptive horizons of the researcher, and so the density of the artifact produced through meaning field analysis is

flexible, not determinate. We experience meaning fields, but reconstruct them through analysis. In the next several pages I will say things like “The meaning field for this speech act is XYZ” but that is sloppy. I should say “the process of meaning field analysis produces the artifact XYZ.” The meaning field that I experience for a given speech act may be more or less populated than the meaning field someone else might experience, and my meaning field may also change in density if new information is made explicit. But once the meaning field has been recollected, usually for analytic purposes, the count of individual elements stabilizes.

For example, if a researcher observed someone saying “I need a cup of coffee” to another interlocuter the researcher might articulate a meaning field {I am addicted to caffeine AND I will get a headache if I do not have caffeine AND coffee has caffeine OR I am sleepy XOR I am frustrated by you this morning and need an excuse to be anywhere else but in your company AND the coffee pot is in another room so it will do} among a host of other possible interpretations depending on the circumstances. (XOR is a term in computer science that means exclusive disjunction, I use it instead of Carspecken’s preferred “hard OR.”) Additional information reduces these possible interpretations. For example, the other interlocutor might say “Are you frustrated with me?” and the original interlocutor might say “Yes. I need to step away for a moment, but I will return and we can finish our discussion when I am not quite so angry.” In doing so, the biological imperative of addiction is ruled out as the primary interpretation. The new meaning field would be {~~I am addicted to caffeine AND I will get a headache if I do not have caffeine AND coffee has caffeine OR I am sleepy XOR~~ I am frustrated by you this morning and need an excuse to be anywhere else but in your company AND the coffee pot is in another room so it will do}.

The mapping is just that AND and OR indicate that the utterances connected by these operators are compossible, existing in the same compossible region in the manifold. Anaphora functions similarly, stitching separable speech acts together within the same universe of thought, fixing the content of p by speaking of "it." p AND q claims that the assertions p and q are within the same universe of thought. One might express this as Brandom does in saying that "what is incompatible with the conjunction Kpq should just be whatever is incompatible with the set $\{p, q\}$" (Brandom, 2019, p. 128). However, as I have defined *set* so loosely as to be merely the content of recollection, the contents between quotation marks, it is a bad kind of circularity to define AND as Brandom does - for my purposes at least - since there is not really a way to smoosh distinct assertions together within a recollection without the notion of conjunction. Because of these concerns, I maintain that conjunction is the claim of compossibility, the idea that two thoughts co-occur within the same normative horizon for a subject, or the simultaneous foregrounding of two assertions who share the same universe of thought. Disjunction, in the inclusive but pre-logical sense, thereby must be interpreted as occurring at the boundary between compossible regions in the manifold, as the pair of assertions may or may not share a universe of thought. Recall that the terms of the vocabulary {inside, outside, boundary} were elaborated from the body-feeling in chapter one. I wish to draw on that same elaboration here to indicate boundaries between compossible regions in the manifold. OR is rather like one's skin. XOR indicates a claim to distinct compossible regions in the manifold, p and q are claimed as being outside one another.

Recent findings from cognitive linguistics (Crain, 2008; Crain & Khlentzos, 2007; Goro & Akiba, n.d.; Jing et al., 2005; Singh, 2008; Singh et al., 2016) suggest that the innate disjunctive operator is inclusive in young children. While I hesitate to rely on notions like

‘biological innateness,’ it does make some sense that a modally robust exclusive disjunction, XOR, would be acquired on some developmental trajectory. As a consequence, we ought to be able to research this acquisition empirically and find the seams between inclusive and exclusive disjunction. If we consider a human before the acquisition of this ability to a human after acquiring this ability, the former does not have a fully formed notion of incompatibility. Incompatibility, in either alethic or deontic normative modalities, requires that the incompatible properties inhere in the same ‘object’. “If A and B are different things, then one can be circular and the other triangular, one red and one green. But one and the same thing cannot have those incompatible properties. A’s being circular and red excludes its being triangular or green. Objects are individuated by such exclusions” (Brandom, 2019, p. 112). This analysis is sufficient for objects, but it does not quite jibe with how meaningful action bifurcates the future so that to do A results in a different manifold than doing B. Brandom’s answer for this lies in his discussion of subjunctive and counterfactual thinking. The development of the pre-logical XOR is entwined with the development of counterfactual and subjunctive thought because doing A precludes the possibility of doing B.

When I first used meaning field analysis to try to understand students’ drawings of quantity, I found that there were some students who seemed to interpret an implicit disjunctive operator in the inclusive sense, while their teachers were presumably interpreting the operator in an exclusive sense. This gave rise to a claim that young learners may experience a denser meaning field for a given communicative act than teachers experience. Learning, then, would be the depopulation of a meaning field. If one assumes an exclusive definition of disjunction, the smallest set of non-trivial elements in the meaning field for the speech act “You can have pizza or ice-cream” has two elements {you can have pizza not ice-cream XOR you can have ice-cream

not pizza}. With an inclusive interpretation of disjunction, the meaning field has an additional element {you can have pizza not ice-cream XOR you can have ice-cream not pizza XOR you can have pizza and ice-cream}. Should the child pile pizza and ice-cream on their plate, the resulting "no" from an authority figure provides an informative signal that reduces the number of possible interpretations of the speech act. The process of information reducing the density of a meaning field is how I define learning.

Modality is a way of extending logic past the basics of conditional, conjunction and disjunction operators. Brandom is concerned with logic in the *alethic modality*, with a vocabulary of *sufficiency, necessity, possibility, impossibility*, and *incompatibility*. He is also interested in logic in the *deontic normative modality*, with a vocabulary of *commitment, entitlement, propriety*, and *incompatibility*. Incompatibility occurs in both modalities in what Brandom calls a *hylomorphism* between expressions in each modality. The first side of the hylomorphism is the *alethic* side that it is *impossible* for incompatible properties to apply to objects one takes as 'the same.' The alethic modality allows for discussions of objects and properties in stable, non-contradictory ways. It allows for talk to turn to the world of objects, and so the world of physical science. For incompatibilities to inhere in an object is to treat that object like it is a subject, but those incompatibilities are explicable in the *deontic normative* sense, not in the alethic modal sense. "It is not that one cannot undertake incompatible commitments, make incompatible assertions. Finding that one has done so is an all-too-common occurrence" (Brandom, 2008, p. 120). My point in saying this is to suggest that the alethic modality is developmentally secondary to the deontic normative modality and so we ought to be able to find empirical evidence of when the alethic modality 'comes online.' There should be a moment where "ought-not" is transformed into "cannot." In the discussion of *hybridized models* that

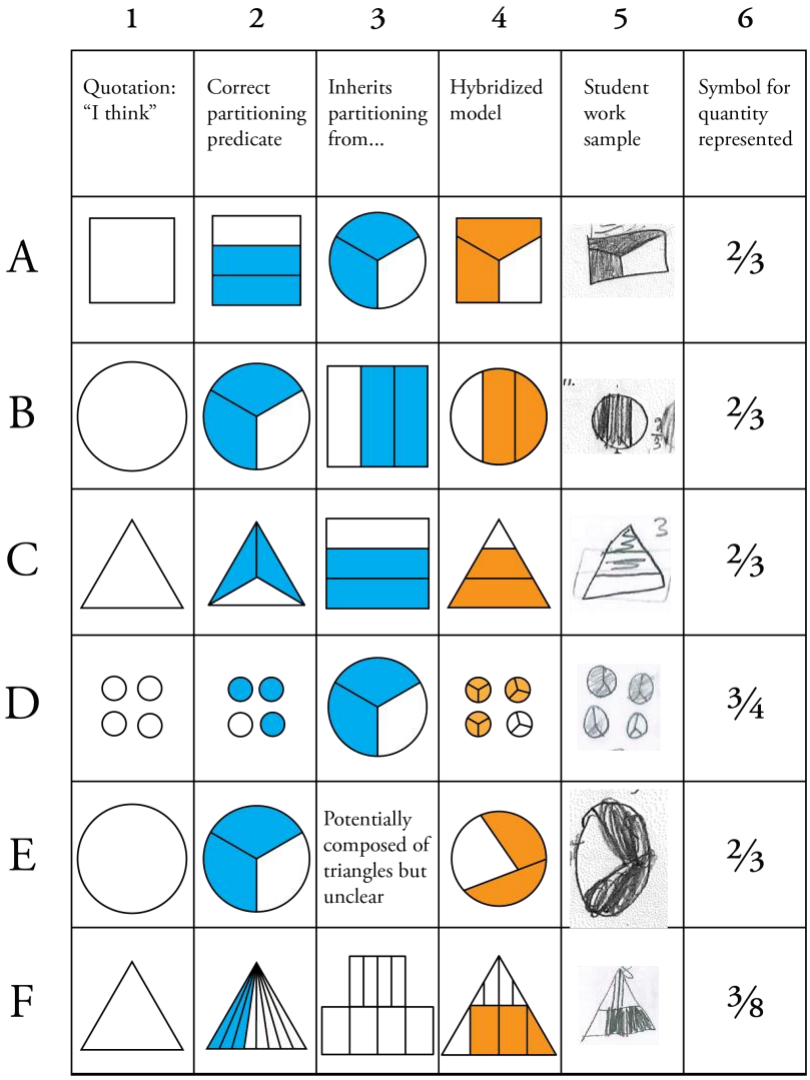
follows, I do not find this seam precisely. Instead, I merely posit that students who work with *hybridized models* have not developed XOR with respect to models of quantity, rather than claiming that such usage indicates that the alethic modality has not been fully developed. That is, I try to classify a set of errors as occurring within the much broader category of mistaking "ought-not" for "cannot."

Hybridized Models

In the context of the ASKTM project, we asked fourth graders to "Write the fractions $\frac{2}{3}, \frac{3}{4}$ and $\frac{3}{8}$ in order from smallest to largest. *Use pictures, words, or symbols to explain.*" In figure 25, I have included cropped student work samples in column 5. The representations of "I think," i.e. circles, rectangles, and triangles, are listed in column 1. In column 2 I have included the correct partitioning predicate, and in column 3 I have drawn the partitioning predicate that has been used erroneously. Column 4 is the resulting hybridized model. To be clear, there were a few repetitions of some of these models, though their overall contribution to the sample space of student work for this particular question was not significant.

Figure 25

Hybridized Representations

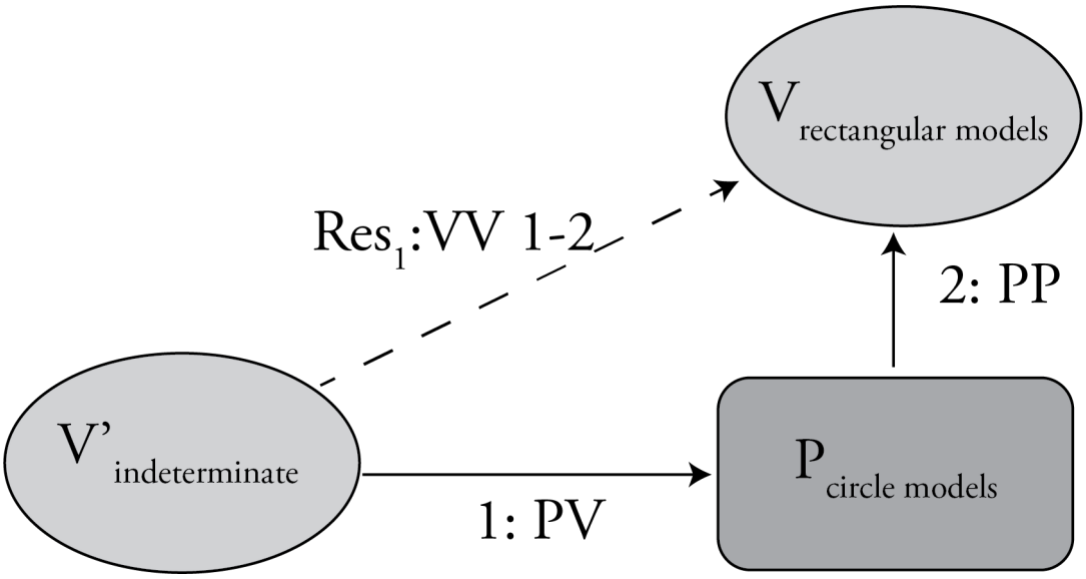


Note. Hybridized models inappropriately deploy the practices-or-abilities associated with one vocabulary to a different vocabulary.

Descriptively, hybridized models occur when the practices-or-abilities associated with one vocabulary are used to make claims over terms from different vocabulary (see figure 26).

Figure 26

MUD for Hybridized Representations



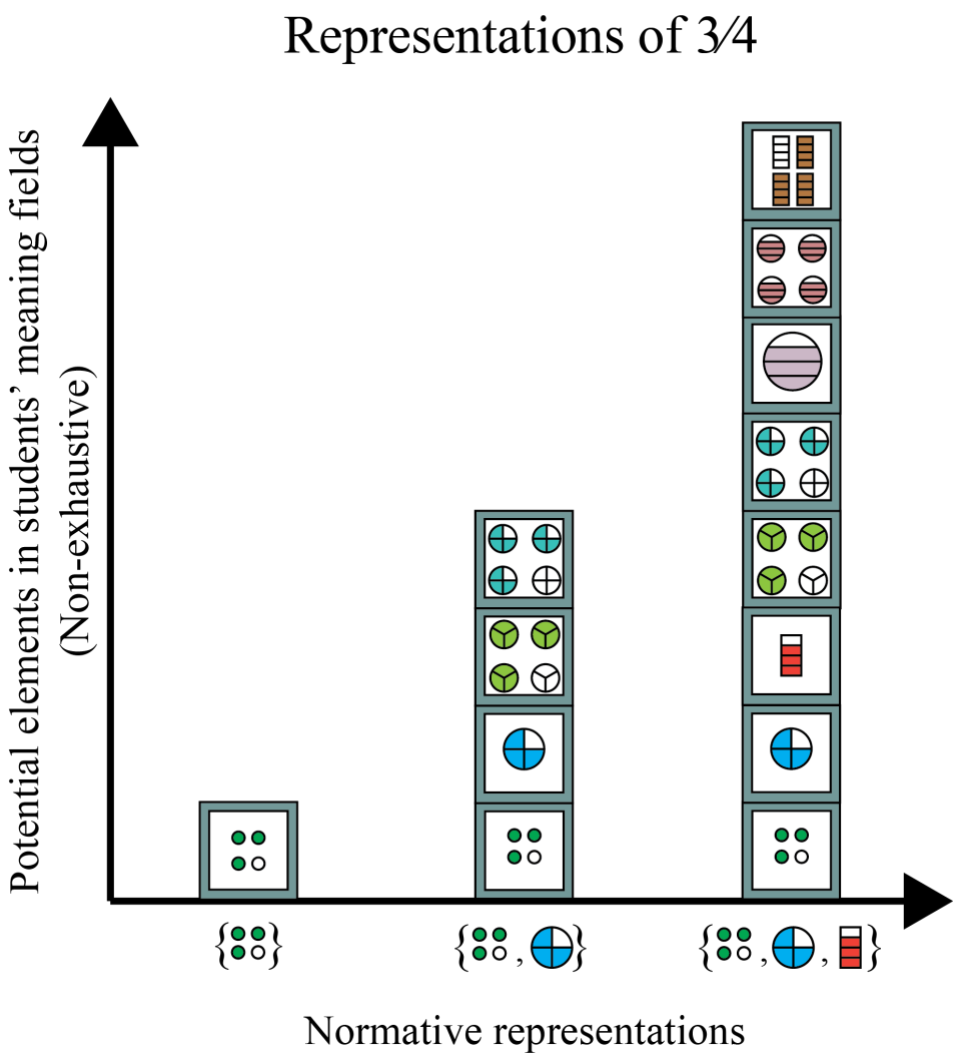
Note. In this MUD for hybridized representations, the practice-or-ability for circle models is used to deploy the vocabulary for rectangular models.

Without the subjunctively sensitive, alethic modal claim that upon choosing which vocabulary to draw on to represent "I think" one must stay within that vocabulary to define the partitioning predicate, the result is that many such hybridizations stack within an individual. Quantitatively, these stacks accrue roughly according to the power set, 2^n, where n is the number of vocabularies one might model a particular quantity in. That is, if a student can model fractions with circle diagrams and rectangular diagrams, their hybridized stack would have 2^2 elements in it (the two correct elements, a radially partitioned rectangular model, and a vertically partitioned circle). In figure 27, I have documented how additional representational vocabularies result in approximately 2^n hybridizations. The grammars of spatial vocabularies are fuzzy, so

“approximately” is the right word choice here. The point of counting these representational failures at all is to indicate the combinatoric heft of confusion. Without knowing what one is permitted to mix and match results in a vastly denser meaning field than what teachers, or others who are not confused, might expect. If a teacher were to say “you may model this quantity with a rectangle OR a circle,” and that OR is inclusive, the result is a vast, if somewhat useless, soup of possibilia compared to what the XOR interpretation would yield.

Figure 27

Combinatorics of Hybridized Representations



Note. Hybridized representations grow at an exponential rate as new vocabularies are introduced. Without determinate negation, the state of confusion can be described through 2^n where *n* is the number of vocabularies in play.

Anaphoric Analysis of Hybridized Models

While the above descriptive analysis is adequate to define hybridized models and indicate the kind of error being made, it does not yet express why that type of error is, in fact, an error. That is, the description allows us to say "you cannot do this thing" it does not answer the question of why that thing ought not be done. To answer that question, I return, again, to anaphora.

What characterizes singular terms from chapter one were their involvement in symmetric substitution inferences. We might substitute "Former President Obama" with "Barack Obama," indicating that those terms belong to an equivalence class of other such terms that are symmetrically intersubstitutable with one another. If numbers were also singular terms, they would be involved in the same kinds of equivalence classes and substitution inferences. Indeed, most of the time they function like singular terms precisely because of their symmetric substitution licenses. Everywhere where one encounters 5 one can substitute 4+1 without deleterious effects.

However, the idea that numbers are singular terms falls somewhat short of modelling quantities spatially, and the anaphoric notion of number that I posited in chapter one does something that the singular term conception does not allow for. Specifically, if the singular term conception is correct then one unit can be represented as a circle or as a square and so those representations would belong in an equivalence class together. The partitioning predicate *ought* to be stable under those symmetric substitutions. That is, when we say "Barack Obama is a former President of the United States" one can substitute in "Former President Obama" without

changing the predicate (□ is a former President of the United States) to obtain "Former President Obama is a former President of the United States."

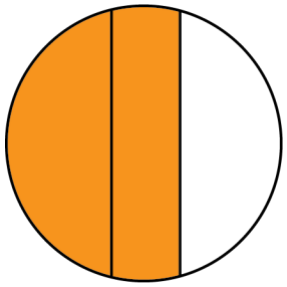
This is not how partitioning predicates work. In the sentence "One unit is partitioned into thirds," the sentence frame is "□ is partitioned into thirds." The contents of this predicate depend on the contents of the term substituted in. If this were not so, then the partitioning predicate for circular models (A3 in figure 25) would apply to the underlying substitution of 'singular terms' when A1 is substituted for B1. Instead, such a substitution yields the hybridized model A4. If "one $\text{unit}_{\text{circle}}$," "one $\text{unit}_{\text{rectangle}}$," and "1" were all members of an equivalence class of symmetrically intersubstitutable singular terms, the hybridized model would preserve good inferences. It does not.

On the other hand, if "one unit" is understood as the anaphoric recollection "I think," the claim to symmetric substitution is not present or implied. While there may be an equivalence class of terms that may all be the referent of "he" (i.e. "men") anaphora does not imply that there are any systematic substitutions between the predicates that apply to those terms. That is, I can say "He was an ok President" and "He was a terrible President" without contradiction because the referent of the anaphoric term is not the same. Anaphoric terms have syntactic sameness without semantic sameness. Moreover, what the "I think" quotes is pure negativity (∅), so there is no grounds to assume that one quotation of "I think" has the same referent as another since what each is quoting is essentially dynamism - each refers to a different act of thought. Consequently, when the "I think" is invoked, the referent ∅ is dynamic while the recollection of that dynamic referent is stable.

Because of this we can state that the error in hybridized models stems from treating spatial quantities (or perhaps I mean numerals) like singular terms rather than anaphoric terms. Of course one might also note that the error results from the claim of equipartitioning collapses if one were to measure the areas involved, but that explanation fetishizes quantity by turning it into an object. In figure 28, I equipartition a circle by measuring equal areas. The resulting pieces are not intersubstitutable so it is not a successful model of $\frac{2}{3}$.

Figure 28

Circle model equipartitioned by area



Note. One *can* vertically equipartition a circle model by area, but such partitions have more limited utility than radial equipartitions.

In chapter 1 I indicated that I was unsure whether the singular term interpretation of number was correct or if the anaphoric notion was better. Hybridized representations offer some evidence to tip those scales towards the anaphoric notion of number as the correct notion. Their illicitness is somewhat inexplicable under the singular term way of thinking but understandable under the anaphoric notion.

Hermeneutic Calculator

While the formulation of the notion of the *Hermeneutic Calculator* in the introduction is a bit grandiose, the embodiment of that calculator is explicated with relatively mundane interests. When I was tasked with doing a fine-grain analysis of student work samples for the ASKTM project, I found that the descriptions I developed were nearly expressions of algorithms written in pseudo-code. I got curious about what it would look like if there were a program that calculated as students do. Such a program, which I imagined as an iPhone app, could be opened by pre-service teachers and explored. I thought it would be neat if there were some way for pre-service teachers to explore models of student thinking in a way that would be accessible, without too much conceptual overhead. That notion blossomed into the unruly proportions documented below as I learned more about incompatibility semantics. It occurred to me that making the vast store of 'all mathematical procedures' one might produce a correct mathematics. The following is somewhere in between those two purposes. I code in SWIFT, the language for iPhone apps, because it is relatively user-friendly and robust and I can imagine other researchers crowd-sourcing examples as a kind of wikiMath. However, I also maintain that this *is* a part of a pragmatist mathematics. It also occurred to me that we, the research team, inevitably made inferences about what a particular student might do with a similar set of numbers. That is, something about the act of precisely writing down what students had done seemed to inevitably lead to questions about what that student might do. I became interested in how we might shore up those inferences.

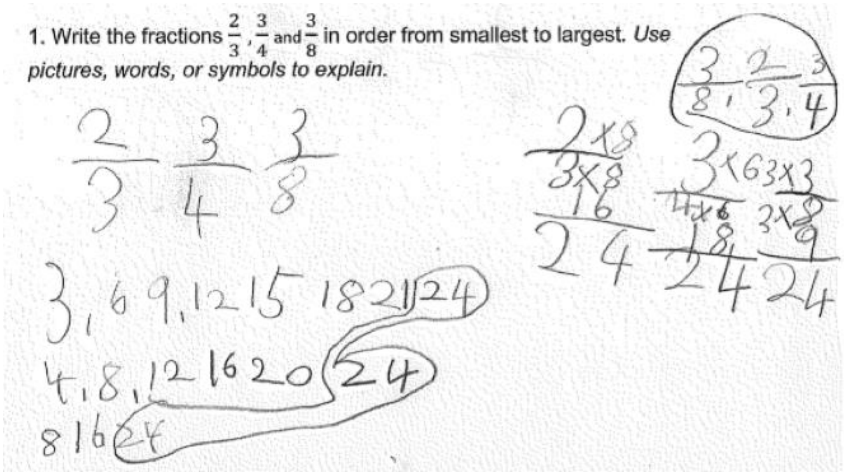
The main problems with shoring up those inferences was that they involved some projection. Moving from a description to a prediction is the basic work of the empirical-analytic knowledge constitutive inference, so the act of making these predictions is essentially an

empirical task. The act of description in pseudo-code was more defined by the hermeneutic-historical interest since it involved trying to make sense of the communicative act of another person. Blending those two interests results in the troubling idea that one might predict what another might say. This is troubling in the sense that one of the foundations of intersubjectivity is the assumption that the other can say "no," and I do not give the students the opportunity to do so. It would be better to have done this work on pre-existing dialogic data like Steffe's (2001) Jason, where more care was taken to ensure that the student's thoughts are being accurately reproduced. That being said, the process for analyzing simple student work is sufficiently challenging to communicate that the simplicity of the exemplars might be a feature rather than a bug.

The basic idea for what I do with the Hermeneutic Calculator is that student work samples like that in figure 29 can be analyzed into a pseudo-code according to the hermeneutic-historical interest, where the point is to understand what the other is communicating. An inference is then established between that student work sample and a machine-readable code that reproduces that sample. A second inference is established between the pseudo-code and a machine-readable version of that pseudo-code that allows for different inputs. The result is a new inference that goes from machine-coding to machine-coding. While this is convoluted, the idea is that I am trying to limit where the projection is happening. It is happening on the machine side, rather than happening on the human side. This allows for empirical-analytic predictions to occur over human reasoning without subsuming human reasoning into mechanical thinking.

Figure 29

Student work sample for projective validity



Note. I use this example to discuss projective validity in the text.

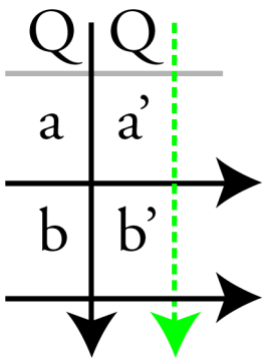
What follows is somewhat overwrought and convoluted because I am imagining that future iterations of the Hermeneutic Calculator could be crowd-sourced. It is too much work for one researcher to generate the whole field of negative mathematics that could then be processed according to incompatibility semantics to yield a functional mathematics. Because of this, I spend a fair amount of effort trying to establish the rules for how code could be added to the Hermeneutic Calculator and how those changes could be documented using MUDs. This effort is explicated in the section on *projective validity*. I then back away from that precision in the next five examples. Before that conversation can occur, I must first relate what *projective inferences* are in general and articulate why their usage is appropriate in this context.

Projective Inferences

Projective inferences are one of the trickier elements of Brandom's inferentialism (2000, p. 124-129). In essence, what a projective inference is relies on notions of sentence frames, Q, along with substituted-for expressions a and b. The idea is that new correct inferences can be formed, provided that they inherit some correctnesses from the family of assertions they are projected from. That is, if substitution inferences from a to a' and b to b' and the inference Qa so Qb are all endorsed, the projective inference Qa' so Qb' *ought* to be good. In general, the sentence frames that I will use involve more than one argument, so while in chapter one I talked of sentence frames like "Q: □ is a dog," in this context I will use α and β to mark the adicities of the sentence frame and speak of $Q(\alpha, \beta)$. Because I am using inferential projection as a research method, it is appropriate that the sentence frames in question have as many adicities as is necessary.

Figure 30

Diagram of a projective inference



Note. The projective inference is diagramed as a green arrow. Its goodness depends on the goodness of the inferences drawn as black arrows.

To construct a simple example, let:

1. $Q(\alpha)$: α is an even number.
2. a: Any number divisible by 2
3. b: 6
4. a': Any number, provided that when the number is divided by 2 the remainder is zero

   b': 8-2
5. Qa so Qb: Any number divisible by 2 is an even number so 6 is an even number.
6. Qa' so Qb': Any number, provided that when the number is divided by 2 the remainder is zero, is an even number, so 8-2 is an even number.

The definition of even numbers supports the inference from a to a'. The inference from b to b' is supported by taking those quantities as symmetrically intersubstitutable. The inference

Qa so Qb could be incorrect but because it is good in this case, the projective inference from Qa' to Qb' will inherit that correctness.

While Brandom (2000) uses projective inferences to account for how competent speakers can produce good inferences and interpret diverse speech acts, his account is less helpful when dealing with inferences between vocabularies. Often a creative act occurs when old truths are expressed in novel vocabularies and projective inferences do not do very well with inter-vocabulary inferences.

My purpose in using projective inferences is to put up some guidelines for the kinds of inferences researchers can make from student work samples. While the normal type of inference that a researcher makes (a so b) is falsifiable in one location, projective inferences are falsifiable in three locations: a so a', b so b', and Qa so Qb. The goodness of the inference Qa' so Qb' is inherited from the goodness of the three inferences it depends on. In the following section, I discuss the notion of *projective validity*.

Projective Validity

In this section, I present a pragmatic approach for conducting a basic kind of empirical research called *model building* in the domain of fractions as a 6-step algorithm, summarized below. While projective validity does not depend on machines or computer code, the particular embodiment of the concept I present utilizes translations between student artifacts, pseudo-code, and computer code to produce a projected inference. In order to explicate a projected inference, the constituent substitution inferences a so a', b so b', Qa so Qb must be endorsed and the sentence frame, Q, must be declared. The goodness of these three inferences is generally, though not certainly, inherited by the projective inference Qa' so Qb'.

Protocol for projective validity

1. Declare $Q(\alpha, \beta)$.
2. Determine $a(\alpha, \beta)$
    a. Declare α
    b. Declare β
3. Generate $b(\alpha, \beta)$
    a. Describe the student work sample (β from $a(\alpha, \beta)$) by reconstructing it in pseudo-code.
    b. Declare α by choosing and justifying new inputs based on the β from the previous step.
    c. β: Input α in pseudo-code
    d. Test inference Qa so Qb.
    e. Adjust pseudo-code until inference is endorsed.
4. Determine $a'(\alpha, \beta)$ and $b'(\alpha, \beta)$
    a. Develop machine code that reconstructs both $a(\alpha, \beta)$ and $b(\alpha, \beta)$.
    b. Test substitution inference a so a'. Does the machine produce a speech act, a', that can be substituted for a?
    c. Test substitution inference b so b'. Does the machine produce a speech act, b', that can be substituted for b?
5. Test the inference Qa' so Qb'. If it is incorrect, repeat steps 1-4 until it is correct. Such an iteration would establish projective validity.
6. On exit from 5, document with Meaning Use Diagram and add the machine code to the Hermeneutic Calculator as a function that algorithmically elaborates the practices-or-

abilities associated with one vocabulary (e.g. ordering whole numbers) into the practices-or-abilities associated with another number (e.g. ordering fractions).

Example 1 of protocol for projective validity

Returning to the student work sample in 29, I now fill in the protocol.

Protocol for projective validity

1. Declare $Q(\alpha,\beta).Q(\alpha,\beta)$: Write the fractions α in order from smallest to largest. *Use pictures, words, or symbols to explain*. β: either student or machine-generated work sample
2. Determine $a(\alpha,\beta)$
    a. Declare α:$\frac{2}{3},\frac{3}{4},\frac{3}{8}$
    b. Declare β: student work sample in 29
3. Generate $b(\alpha,\beta)$
    a. Describe the student work sample from $a(\alpha,\beta)$) by reconstructing it in pseudo-code: Common denominator, includes list of multiples to aide in finding the LCM, shows multiplication by the appropriate factor to obtain LCM, includes equivalent fractions, reverts to original form for ordering
    b. Declare α by choosing and justifying new inputs based on the β from the previous step: $\frac{5}{8},\frac{2}{3},\frac{1}{2}$
    c. Declare β as the justification of α in the previous step: $\frac{5}{8},\frac{2}{3},\frac{1}{2}$ ought to be appropriate numerical substitutions for this student's work because the original numerators and denominators are small whole numbers. It is unclear from the

sample how willing the student would be to actually write out all of the necessary multiples so choosing some denominators that are not relatively prime would be best. Given the new numbers the student would probably respond by finding a common denominator by listing multiples of each denominator, identifying the least common multiple, writing out the equivalent fractions in order by the magnitude of the new numerators, and reverting to the original fractions when writing the ordered fractions.

d. Test inference Qa so Qb. This is the first falsifiable inference. Is the description of the student work accurate and sufficient? Does the inference regarding how the student might respond to different numbers good? [13]

e. Adjust pseudo-code and description until the inference is endorsed.

4. Determine $a'(\alpha, \beta)$ and $b'(\alpha, \beta)$

a. Develop machine code that reconstructs both $a(\alpha, \beta)$ and $b(\alpha, \beta)$.

```
import UIKit
var nB1 = 3
var nB2 = 4
var nB3 = 8
var nA1 = 2
```

[13] My claim is not that these inferences, as I explicate them, are actually good or that the description is sufficient. Rather, my point is that this analysis allows for such contentions to be made explicit.

```
var nA2 = 3

var nA3 = 3

var multipliernB1 = nB1

var multipliernB2 = nB2

var multipliernB3 = nB3

var modAB = 1

var modAC = 1

var modABC = 1


var multiplesofnB1 = ["\(nB1)"]

while modABC != 0 {

    multipliernB1 = nB1 + multipliernB1

    modAB = multipliernB1 % nB2

    modAC = multipliernB1 % nB3

    modABC = modAB + modAC

    multiplesofnB1.append("\(multipliernB1)")


}

print(multiplesofnB1)



modAB = 1

modAC = 1
```

```
modABC = 1
var multiplesofnB2 = ["\(nB2)"]

while modABC != 0 {
    multipliernB2 = nB2 + multipliernB2
    modAB = multipliernB2 % nB1
    modAC = multipliernB2 % nB3
    modABC = modAB + modAC
    multiplesofnB2.append("\(multipliernB2)")

}
print(multiplesofnB2)

modAB = 1
modAC = 1
modABC = 1
var multiplesofnB3 = ["\(nB3)"]

while modABC != 0 {
    multipliernB3 = nB3 + multipliernB3
    modAB = multipliernB3 % nB1
    modAC = multipliernB3 % nB2
    modABC = modAB + modAC
```

```
    multiplesofnB3.append("\(multipliernB3)")

}
print(multiplesofnB3)

var LCM = 0
if multipliernB1==multipliernB2 && multipliernB1==multipliernB3 {
    print("LCM = \(multipliernB1)")
    LCM = multipliernB1
}

var newnA1 = nA1*LCM/nB1
var newnA2 = nA2*LCM/nB2
var newnA3 = nA3*LCM/nB3

print("\(nA1)/\(nB1) -> \(nA1)*\(LCM/nB1)/\(nB1)*\(LCM/nB1)=\(nA1*LCM/n
B1)/\(LCM)")
print("\(nA2)/\(nB2) -> \(nA2)*\(LCM/nB2)/\(nB2)*\(LCM/nB2)=\(nA2*LCM/n
B2)/\(LCM)")
print("\(nA3)/\(nB3) -> \(nA3)*\(LCM/nB3)/\(nB3)*\(LCM/nB3)=\(nA3*LCM/n
B3)/\(LCM)")

if newnA1<newnA2 && newnA1<newnA3 && newnA2<newnA3 {
```

```
    print("\(newnA1)/\(LCM) < \(newnA2)/\(LCM) < \(newnA3)/\(LCM)")
    print("So \(nA1)/\(nB1) < \(nA2)/\(nB2) < \(nA3)/\(nB3)")
}
if newnA1<newnA2 && newnA1<newnA3 && newnA3<newnA2 {
    print("\(newnA1)/\(LCM) < \(newnA3)/\(LCM) < \(newnA2)/\(LCM)")
    print("So \(nA1)/\(nB1) < \(nA3)/\(nB3) < \(nA2)/\(nB2)")
}



if newnA2<newnA1 && newnA2<newnA3 && newnA1<newnA3 {
    print("\(newnA2)/\(LCM) < \(newnA1)/\(LCM) < \(newnA3)/\(LCM)")
    print("So \(nA2)/\(nB2) < \(nA1)/\(nB3) < \(nA3)/\(nB3)")
}
if newnA2<newnA1 && newnA2<newnA3 && newnA3<newnA1 {
    print("\(newnA2)/\(LCM) < \(newnA3)/\(LCM) < \(newnA1)/\(LCM)")
     print("So \(nA2)/\(nB2) < \(nA3)/\(nB3) < \(nA1)/\(nB1)")
}

if newnA3<newnA1 && newnA3<newnA2 && newnA1<newnA2 {
    print("\(newnA3)/\(LCM) < \(newnA1)/\(LCM) < \(newnA2)/\(LCM)")
    print("So \(nA3)/\(nB3) < \(nA1)/\(nB1) < \(nA2)/\(nB2)")
}
if newnA3<newnA1 && newnA3<newnA2 && newnA2<newnA1 {
```

```
    print("\(newnA3)/\(LCM) < \(newnA2)/\(LCM) < \(newnA1)/\(LCM)")
    print("So \(nA3)/\(nB3) < \(nA2)/\(nB2) < \(nA1)/\(nB1)")
}
```

b. Test substitution inference a so a'. Does the machine produce a speech act, a', that can be substituted for a? The machine code generates the following - is the substitution a so a' good? (refer to figure 29)

["3", "6", "9", "12", "15", "18", "21", "24"]

["4", "8", "12", "16", "20", "24"]

["8", "16", "24"]

LCM = 24

2/3 -> 2*8/3*8=16/24

3/4 -> 3*6/4*6=18/24

3/8 -> 3*3/8*3=9/24

9/24 < 16/24 < 18/24

So 3/8 < 2/3 < 3/4

c. Test substitution inference b so b'. Does the machine produce a speech act, b', that can be substituted for b? Given the inputs $\frac{5}{8}, \frac{2}{3}, \frac{1}{2}$ the machine produces the following response - is the substitution inference b so b' good?

["8", "16", "24"]

["3", "6", "9", "12", "15", "18", "21", "24"]

["2", "4", "6", "8", "10", "12", "14", "16", "18", "20", "22", "24"]

LCM = 24

5/8 -> 5*3/8*3=15/24

2/3 -> 2*8/3*8=16/24

1/2 -> 1*12/2*12=12/24

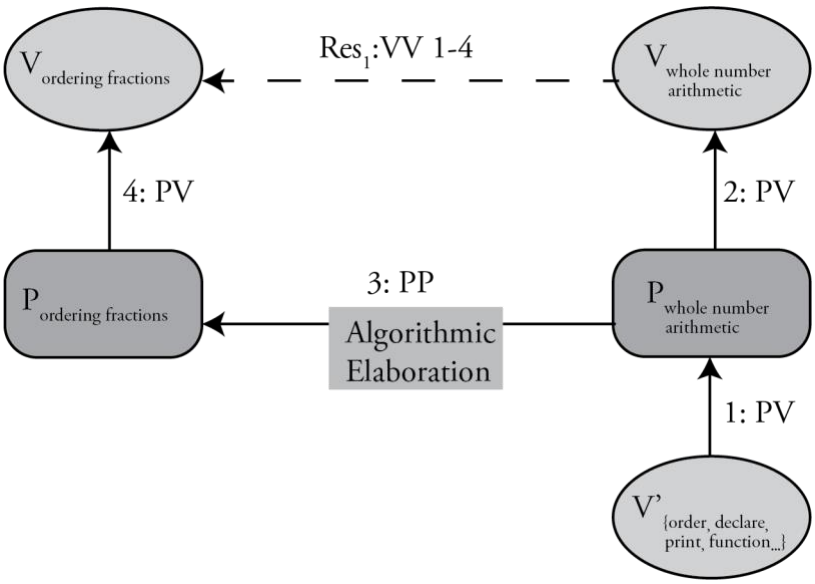
12/24 < 15/24 < 16/24

So 1/2 < 5/8 < 2/3

5. Test the inference Qa' so Qb'. Does the machine answer like we suspect the student might if given new numbers? If it is incorrect, repeat steps 1-4 until it is correct. Such iterations would eventually establish projective validity.
6. On exit from 5, document with Meaning Use Diagram (see figure 31 and add the machine code to the Hermeneutic Calculator as a function that algorithmically elaborates the practices-or-abilities associated with one vocabulary (e.g. ordering whole numbers) into the practices-or-abilities associated with another number (e.g. ordering fractions).

Figure 31

Resulting MUD



Note. This is the MUD that results from the analytic process.

Actually following this full protocol and documenting each step is somewhat onerous, and I sense a risk that the details obfuscate the point. I summarize the protocol in table 4.

Table 4

Specific projective inference

$Q(\alpha, \beta)$: Write the fractions α in order from smallest to largest.		
Use pictures, words, or symbols to explain. β: work sample		
	α	β

aa:	$\frac{2}{3}, \frac{3}{4}, \frac{3}{8}$	Student Work Sample
b:	New numbers	how might the student respond to new numbers
a':	$\frac{5}{8}, \frac{2}{3}, \frac{1}{2}$	Machine Work Sample
b':	New numbers	Machine Work Sample

Note: This is an example of a projective inference to get from student work samples to machine work samples.

Discussion of projective validity

Projective validity is a species of normative validity in general. It is a kind of halfway point between a generic material inference, whose goodness determines the content of the terms caught up in it, and formal inferences which are good by virtue of their form. The paradigmatic example of formal inferences is modus ponens. An example of modus ponens is "If it is raining then the streets will be wet. It is raining, so the streets will be wet." Generically, modus ponens has the form if a then b, a so b. Projective inferences inherit their goodness from the three material inferences that must be established before the projection, but the result is no longer a material inference. It is a formal inference whose goodness depends on the material/circumstantial goodness of its constituent material inferences. Projective inferences are not *necessarily* good. But for them to be good, the rules governing the subsentential elements must be extendible into new material circumstances. This is the commitment to normativity.

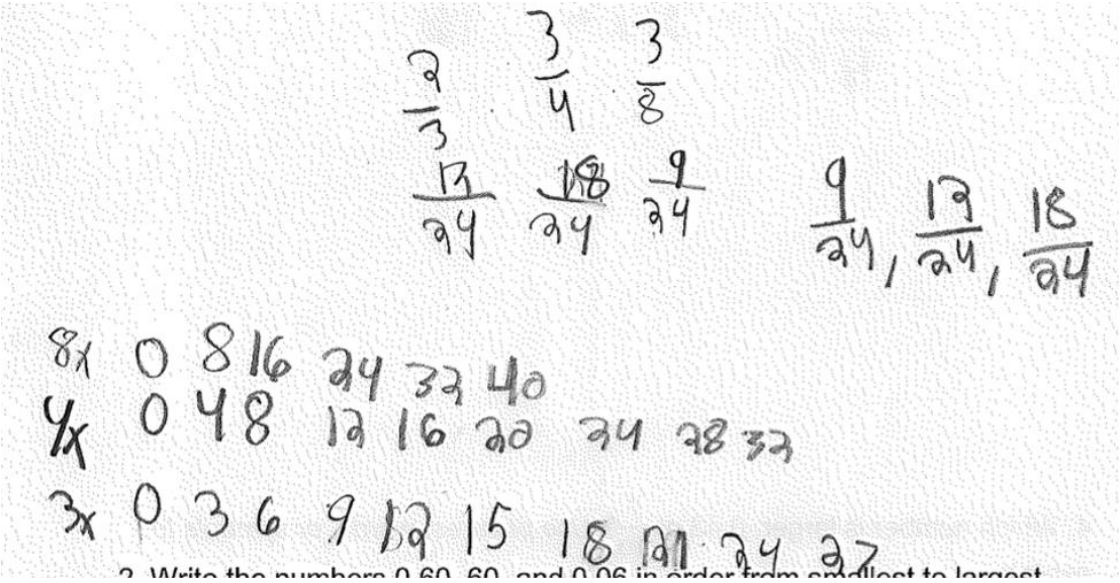
It may be worth noting that I took some pains to make the content of the projection is mechanical in this example by using SWIFT. My point is less about the programming side and more about how we might shore up the everyday inferences that mathematics educators make.

Student Work Examples

In the following examples, I am going to analyze student work samples according to the above protocols, but I will just summarize these analyses to make them a little easier to follow.

Figure 32

Example 2



Note. Student work sample 2

Description: Common denominator, includes list of multiples to aide in finding the LCM, shows multiplication by the appropriate factor to obtain LCM, includes equivalent fractions, does not reduce fractions

New inputs: It makes sense to keep the new inputs small whole numbers with at most one pair of denominators relatively prime to keep the lists of multiples small. Try $\frac{5}{6}, \frac{1}{3}, \frac{1}{2}$.

Given inputs: $\frac{2}{3}, \frac{3}{4}, \frac{3}{8}$ the machine produces the following:

[0, 3, 6, 9, 12, 15, 18, 21, 24, 27]

[0, 4, 8, 12, 16, 20, 24, 28, 32]

[0, 8, 16, 24, 32, 40]

The least common multiple is: 24

2/3 -> 2*8/3*8=16/24

3/4 -> 3*6/4*6=18/24

3/8 -> 3*3/8*3=9/24

9/24 < 16/24 < 18/24

So 3/8 < 2/3 < 3/4

Given inputs $\frac{5}{6}, \frac{1}{3}, \frac{1}{2}$, the machine produces:

[0, 6, 12, 18, 24]

[0, 3, 6, 9, 12, 15, 18]

[0, 2, 4, 6, 8, 10, 12, 14]

The least common multiple is: 12

5/6 -> 5*2/6*2=10/12

1/3 -> 1*4/3*4=4/12

1/2 -> 1*6/2*6=6/12

4/12 < 6/12 < 10/12

So 1/3 < 1/2 < 5/6

Code used to produce those two outputs:

```
import UIKit
    // given fractions nA1/nB1 (standing for "above" and "below")
    var nB1 = 3
    var nB2 = 4
    var nB3 = 8
    var nA1 = 2
    var nA2 = 3
    var nA3 = 3
    var multipliernB1 = nB1
    var multipliernB2 = nB2
    var multipliernB3 = nB3
    // modAB is A% B, or the remainder of A/B. Setting to 1 initially, but it doesn't matter so long
as it is not set to zero.
    var modAB = 1
    var modAC = 1
    // modABC is an integer, modAB+modAC, if both are non-zero then their sum is non-zero. Th
e "while" loop continues until both are A%B and A%C are zero.
    var modABC = 1


    var multiplesofnB1: Set = [0, nB1]
```

```
var maxDenominators: Int = max(nB2, nB3) + 1
while multiplesofnB1.count <= maxDenominators {

    multipliernB1 = nB1 + multipliernB1
    modAB = multipliernB1 % nB2
    modAC = multipliernB1 % nB3
    modABC = modAB + modAC
    multiplesofnB1 = multiplesofnB1.union([multipliernB1])

}
let multiplesofnB1sorted = multiplesofnB1.sorted()
print(multiplesofnB1sorted)



modAB = 1
modAC = 1
modABC = 1
var multiplesofnB2: Set = [0, nB2]

maxDenominators = max(nB1, nB3)
while multiplesofnB2.count <= maxDenominators {

    multipliernB2 = nB2 + multipliernB2
```

```
    multiplesofnB2 = multiplesofnB2.union([multipliernB2])
}
let multiplesofnB2sorted = multiplesofnB2.sorted()
print(multiplesofnB2sorted)

modAB = 1
modAC = 1
modABC = 1
var multiplesofnB3: Set = [0, nB3]
maxDenominators = max(nB1, nB2) + 1
while multiplesofnB3.count <= maxDenominators {

    multipliernB3 = nB3 + multipliernB3

    multiplesofnB3 = multiplesofnB3.union([multipliernB3])

}
let multiplesofnB3sorted = multiplesofnB3.sorted()
print(multiplesofnB3sorted)


let LCM1 =  multiplesofnB2.intersection(multiplesofnB1)
```

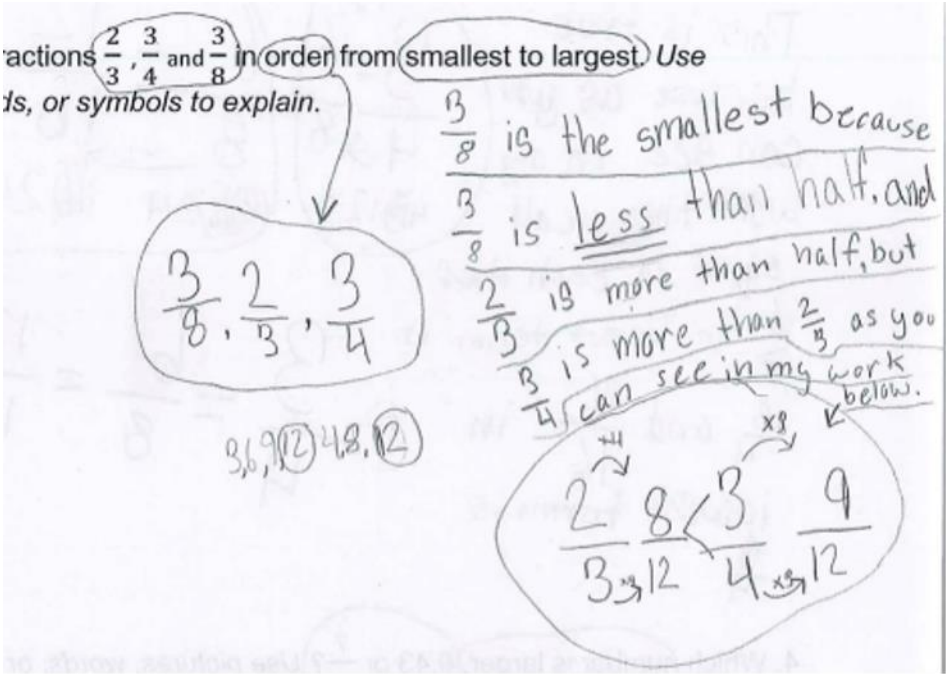
```
    let LCM2 = LCM1.intersection(multiplesofnB3)
    let LCM3 = LCM2.subtracting([0])
    let LCM: Int = Int(LCM3.first ?? 0)

    print("The least common multiple is: \(LCM)")




    var newnA1 = nA1*LCM/nB1
    var newnA2 = nA2*LCM/nB2
    var newnA3 = nA3*LCM/nB3

    print("\(nA1)/\(nB1) -> \(nA1)*\(LCM/nB1)/\(nB1)*\(LCM/nB1)=\(nA1*LCM/nB1)/\(LCM)"
)
    print("\(nA2)/\(nB2) -> \(nA2)*\(LCM/nB2)/\(nB2)*\(LCM/nB2)=\(nA2*LCM/nB2)/\(LCM)"
)
    print("\(nA3)/\(nB3) -> \(nA3)*\(LCM/nB3)/\(nB3)*\(LCM/nB3)=\(nA3*LCM/nB3)/\(LCM)"
)

    if newnA1<newnA2 && newnA1<newnA3 && newnA2<newnA3 {
        print("\(newnA1)/\(LCM) < \(newnA2)/\(LCM) < \(newnA3)/\(LCM)")
        print("So \(nA1)/\(nB1) < \(nA2)/\(nB2) < \(nA3)/\(nB3)")
```

```
}
if newnA1<newnA2 && newnA1<newnA3 && newnA3<newnA2 {
    print("\(newnA1)/\(LCM) < \(newnA3)/\(LCM) < \(newnA2)/\(LCM)")
    print("So \(nA1)/\(nB1) < \(nA3)/\(nB3) < \(nA2)/\(nB2)")
}



if newnA2<newnA1 && newnA2<newnA3 && newnA1<newnA3 {
    print("\(newnA2)/\(LCM) < \(newnA1)/\(LCM) < \(newnA3)/\(LCM)")
    print("So \(nA2)/\(nB2) < \(nA1)/\(nB3) < \(nA3)/\(nB3)")
}
if newnA2<newnA1 && newnA2<newnA3 && newnA3<newnA1 {
    print("\(newnA2)/\(LCM) < \(newnA3)/\(LCM) < \(newnA1)/\(LCM)")
    print("So \(nA2)/\(nB2) < \(nA3)/\(nB3) < \(nA1)/\(nB1)")
}

if newnA3<newnA1 && newnA3<newnA2 && newnA1<newnA2 {
    print("\(newnA3)/\(LCM) < \(newnA1)/\(LCM) < \(newnA2)/\(LCM)")
    print("So \(nA3)/\(nB3) < \(nA1)/\(nB1) < \(nA2)/\(nB2)")
}
if newnA3<newnA1 && newnA3<newnA2 && newnA2<newnA1 {
    print("\(newnA3)/\(LCM) < \(newnA2)/\(LCM) < \(newnA1)/\(LCM)")
```

```
    print("So \(nA3)/\(nB3) < \(nA2)/\(nB2) < \(nA1)/\(nB1)")
}
```

Figure 33

Example 3



Note. Student work sample 3.

Description: Common denominator, benchmark numbers, includes list of multiples to aide in finding the LCM, shows one set of common denominator calculations to compare $\frac{2}{3}$ and $\frac{3}{4}$, benchmark number of $\frac{1}{2}$ to determine the relative position of $\frac{3}{8}$

New inputs: It makes sense to keep the new inputs small whole numbers. Because this student does not rely on making lists of multiples, it might work to use denominators that are relatively prime. Try $\frac{1}{3}, \frac{3}{5}, \frac{3}{8}$.

Given inputs: $\frac{2}{3}, \frac{3}{4}, \frac{3}{8}$ the machine produces the following:

2/3 is greater than 1/2

3/4 is greater than 1/2

3/8 is less than 1/2

2/3 -> 8/12

3/4 -> 9/12

3/8 < 2/3 < 3/4

Given inputs $\frac{1}{3}, \frac{3}{5}, \frac{3}{8}$, the machine produces:

1/3 is less than 1/2

3/5 is greater than 1/2

3/8 is less than 1/2

1/3 -> 8/24

3/8 -> 9/24

1/3 < 3/8 < 3/5

Code used to produce those two outputs (toggle initial group of “let frac1...” and untoggle the next group to produce the second output):

```
import UIKit


class Fractions {
    let name: String
    var numerator: Int
```

```
    var denominator: Int
    init(name: String, numerator: Int, denominator: Int) {
        self.name = name
        self.numerator = numerator
        self.denominator = denominator
    }
}

let frac1 = Fractions(name: "2/3", numerator: 2, denominator: 3)
let frac2 = Fractions(name: "3/4", numerator: 3, denominator: 4)
let frac3 = Fractions(name: "3/8", numerator: 3, denominator: 8)

//let frac1 = Fractions(name: "1/3", numerator: 1, denominator: 3)
//let frac2 = Fractions(name: "3/5", numerator: 3, denominator: 5)
//let frac3 = Fractions(name: "3/8", numerator: 3, denominator: 8)


var frac1A = frac1
var frac2A = frac2
var frac3A = frac3
```

```
var fractions: [Fractions] = [frac1, frac2, frac3]


var lessThanHalf: [Fractions]
var greaterThanHalf: [Fractions]
lessThanHalf = []
greaterThanHalf = []


for fracs in fractions {
if Double(fracs.numerator)/Double(fracs.denominator) <= 0.5 {
    lessThanHalf.append(fracs)
        print("\(fracs.name) is less than 1/2")
} else {
    if Double(fracs.numerator)/Double(fracs.denominator) >= 0.5 {
        greaterThanHalf.append(fracs)
        print("\(fracs.name) is greater than 1/2")
    }
    }
}




var modAB: Int
func gcd(a: inout Fractions, b: inout Fractions) {
```

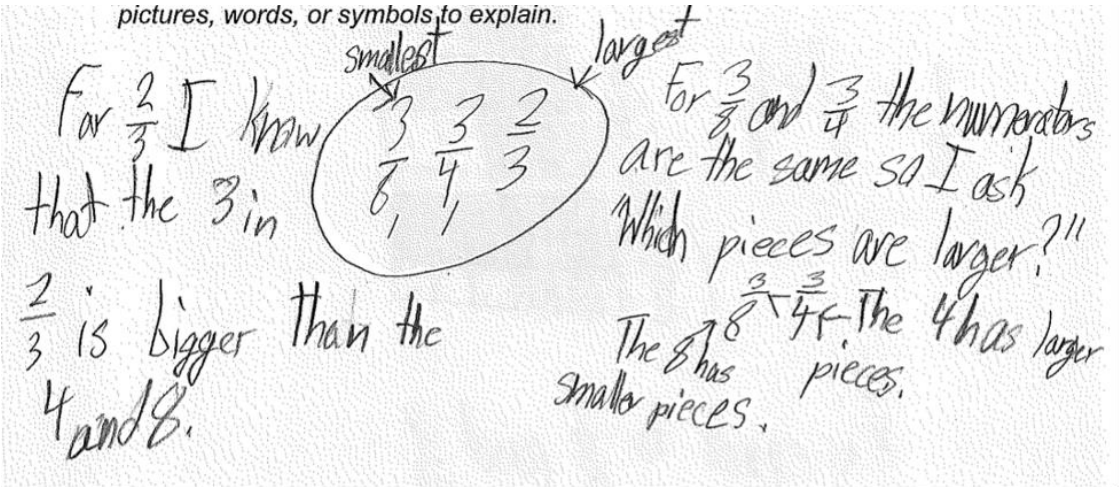
```
   var i: Int = 1
   var Amultiplier: Int = 0
    var Bmultiplier: Int = 0
    var LCMA: Int = 0
    var modAB: Int = 1
    while modAB != 0 {
        i = i + 1
        LCMA = a.denominator * i
        modAB = LCMA % b.denominator
        Amultiplier = LCMA / a.denominator
        Bmultiplier = LCMA / b.denominator
    }
a.denominator = a.denominator * Amultiplier
a.numerator = a.numerator * Amultiplier
b.denominator = b.denominator * Bmultiplier
b.numerator = b.numerator * Bmultiplier
//    return (a, b)
}


//var newA = gcd(a: &frac1A, b: &frac2A)
//print("\(frac1.name) -> \(frac1A.numerator)/\(frac1A.denominator)")
```

```
//if the count of the less than halves is bigger than 1, then a GCD algorithm makes sense. Otherw
ise, the count of the ones greater than a half ought to be bigger than 1, so the GCD algorithm sho
uld run over the others.

var countGreater = greaterThanHalf.count
var countLess = lessThanHalf.count
var fractionsNew: [Fractions]



if countGreater == 2 {
    var greaterThanHalfA: Fractions = greaterThanHalf[0]
    var greaterThanHalfB: Fractions = greaterThanHalf[1]
    gcd(a: &greaterThanHalfA, b: &greaterThanHalfB)
    print("\(greaterThanHalfA.name) -> \(greaterThanHalfA.numerator)/\(greaterThanHalfA.deno
minator)")
    print("\(greaterThanHalfB.name) -> \(greaterThanHalfB.numerator)/\(greaterThanHalfB.deno
minator)")
    fractionsNew = [greaterThanHalfA, greaterThanHalfB]
    let fracSortedByNumerator = fractionsNew.sorted(by: { $0.numerator < $1.numerator})
    print("\(lessThanHalf[0].name) < \(fracSortedByNumerator[0].name) < \(fracSortedByNumera
tor[1].name)")

} else {
```

```
    if countLess == 2 {
    var lessThanHalfA = lessThanHalf[0]
    var lessThanHalfB = lessThanHalf[1]
    gcd(a: &lessThanHalfA, b: &lessThanHalfB)
    print("\(lessThanHalfA.name) -> \(lessThanHalfA.numerator)/\(lessThanHalfA.denominator)"
)
        print("\(lessThanHalfB.name) -> \(lessThanHalfB.numerator)/\(lessThanHalfB.denominator
)")
        fractionsNew = [lessThanHalfA, lessThanHalfB]
        let fracSortedByNumerator = fractionsNew.sorted(by: { $0.numerator < $1.numerator})
        print("\(fracSortedByNumerator[0].name) < \(fracSortedByNumerator[1].name) < \(greater
ThanHalf[0].name)")

    } else {
        print("IDK")
    }
}
//print("\(newA.name) -> \(newA.numerator)/\(newA.denominator)")



let fracSortedByDenominator = fractions.sorted(by: { $0.denominator < $1.denominator})
```

Figure 34

Example 4



Note. Student work sample 4.

Description: Correctly states that if two fractions have the same numerator, the one with the smaller denominator is larger than the other. “The smaller the denominator with the same numerator is larger.”

New inputs: It makes sense to keep the new inputs small whole numbers. Because this student does not rely on making lists of multiples, it might work to use denominators that are relatively prime. Try $\frac{2}{3}, \frac{4}{5}, \frac{4}{8}$.

Given inputs: $\frac{2}{3}, \frac{3}{4}, \frac{3}{8}$ the machine produces the following:

3/8 < 3/4 < 2/3

For 2/3 I know that the 3 in 2/3 is bigger than the 4 and 8. For 3/4 and 3/8, the numerators are the same so I ask, which piece is larger? 3/8 < 3/4 because the 8 has smaller pieces. The 4 has larger pieces.

Given inputs $\frac{2}{3}, \frac{4}{5}, \frac{4}{8}$, the machine produces:

4/8 < 4/5 < 2/3

For 2/3 I know that the 3 in 2/3 is bigger than the 5 and 8. For 4/5 and 4/8, the numerators are the same so I ask, which piece is larger? 4/8 < 4/5 because the 8 has smaller pieces. The 5 has larger pieces.

Code used to produce those two outputs (toggle initial group of “let frac1...” and untoggle the next group to produce the second output):

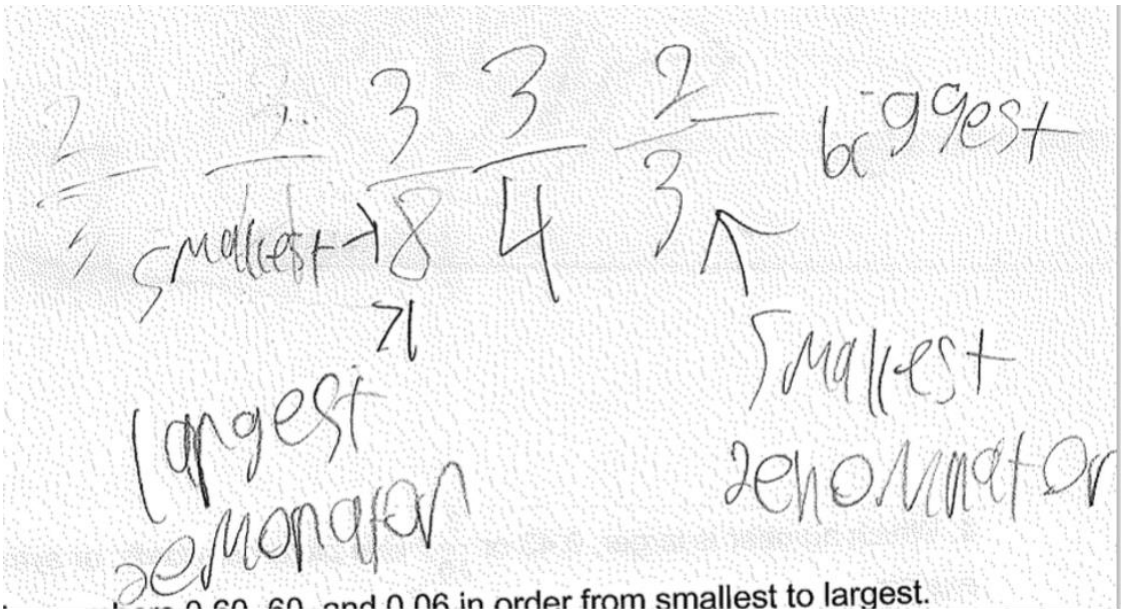
```
import UIKit


class Fractions {
    let name: String
    let numerator: Int
    let denominator: Int
    init(name: String, numerator: Int, denominator: Int) {
        self.name = name
        self.numerator = numerator
        self.denominator = denominator
    }
}


let frac1 = Fractions(name: "2/3", numerator: 2, denominator: 3)
let frac2 = Fractions(name: "3/4", numerator: 3, denominator: 4)
```

```
let frac3 = Fractions(name: "3/8", numerator: 3, denominator: 8)

//let frac1 = Fractions(name: "2/3", numerator: 2, denominator: 3)
//let frac2 = Fractions(name: "4/5", numerator: 4, denominator: 5)
//let frac3 = Fractions(name: "4/8", numerator: 4, denominator: 8)

var fractions: [Fractions] = [frac1, frac2, frac3]

let sortedNumerators = fractions.sorted(by: { $0.numerator < $1.numerator })
// sorts by the size of the numerators (small to large). So in original fractions would yield 2/3, 3/4
, 3/8
//print("\(sortedNumerators[0].name) < \(sortedNumerators[1].name) < \(sortedNumerators[2].na
me)")

let sortedDenominatorsBackwards = fractions.sorted(by: { $0.denominator > $1.denominator })
// sorts by the size of the denominators (large to small)
//print("\(sortedDenominatorsBackwards[0].name) < \(sortedDenominatorsBackwards[1].name)
< \(sortedDenominatorsBackwards[2].name)")
```

```
let sortedDenominators = fractions.sorted(by: { $0.denominator < $1.denominator })
// sorts by the size of the denominators (smallest denominator to largest denominator)
//print("\(sortedDenominators[0].name) < \(sortedDenominators[1].name) < \(sortedDenominator
s[2].name)")


// declare that the largest is the "largest"/last element from the fractions sortedDenominatorsBack
wards
// declare that the smaller of the two remaining should be the one with the largest denominator, th
e first element of sortedDenominatorsBackwards
var fractionsSorted = [sortedDenominatorsBackwards[0], sortedDenominatorsBackwards[1], sort
edDenominatorsBackwards[2]]
print("\(fractionsSorted[0].name) < \(fractionsSorted[1].name) < \(fractionsSorted[2].name)")


print("For \(fractionsSorted[2].name) I know that the \(fractionsSorted[2].denominator) in \(fracti
onsSorted[2].name) is bigger than the \(fractionsSorted[1].denominator) and \(fractionsSorted[0].
denominator). For \(fractionsSorted[1].name) and \(fractionsSorted[0].name), the numerators are
the same so I ask, which piece is larger? \(fractionsSorted[0].name) < \(fractionsSorted[1].name)
because the \(fractionsSorted[0].denominator) has smaller pieces. The \(fractionsSorted[1].deno
minator) has larger pieces.")
```

Figure 35

Example 5



Note. Student work sample 5.

Description: incorrectly states that the larger the denominator the smaller the fraction.

New inputs: It makes sense to keep the new inputs small whole numbers. The denominators ought not be equal or their algorithm breaks down entirely (discussed this example in chapter 2). Try $\frac{1}{2}, \frac{2}{3}, \frac{3}{4}$.

Given inputs: $\frac{2}{3}, \frac{3}{4}, \frac{3}{8}$ the machine produces the following:

3/8 < 3/4 < 2/3 because 8 > 4 > 3

Given inputs $\frac{1}{2}, \frac{2}{3}, \frac{3}{4}$, the machine produces:

3/4 < 2/3 < 1/2 because 4 > 3 > 2

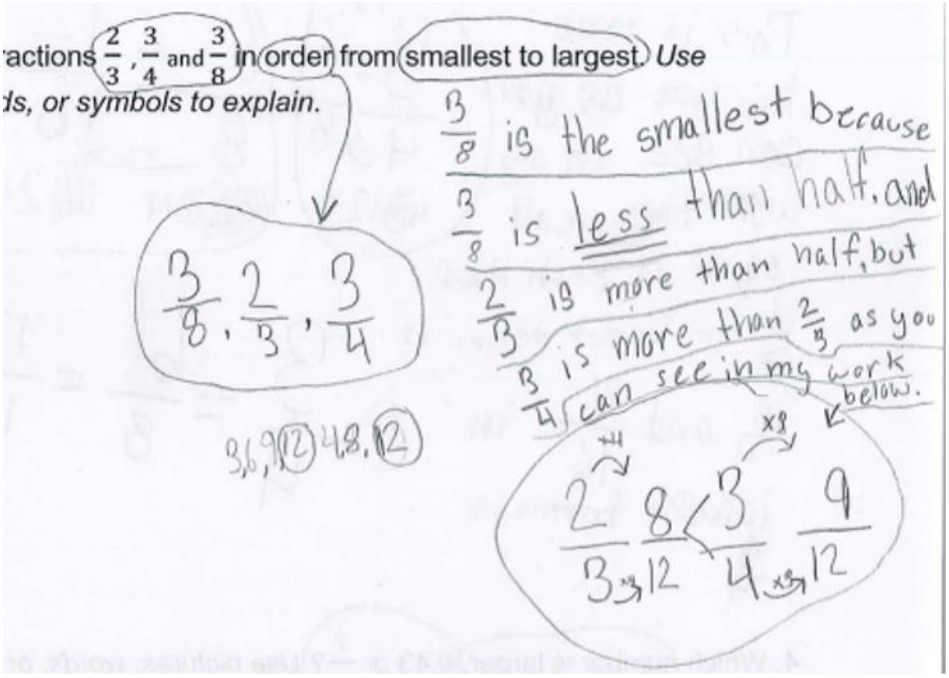
Code used to produce those two outputs (toggle initial group of “let frac1...” and untoggle the next group to produce the second output):

```
import UIKit

class Fractions {
    let name: String
    var numerator: Int
    var denominator: Int
    init(name: String, numerator: Int, denominator: Int) {
        self.name = name
        self.numerator = numerator
        self.denominator = denominator
    }
}

let frac1 = Fractions(name: "2/3", numerator: 2, denominator: 3)
let frac2 = Fractions(name: "3/4", numerator: 3, denominator: 4)
let frac3 = Fractions(name: "3/8", numerator: 3, denominator: 8)

//let frac1 = Fractions(name: "1/2", numerator: 1, denominator: 2)
//let frac2 = Fractions(name: "2/3", numerator: 2, denominator: 3)
//let frac3 = Fractions(name: "3/4", numerator: 3, denominator: 4)
```

```
let fractions = [frac1, frac2, frac3]

let fracSortedByDenominator = fractions.sorted(by: { $0.denominator > $1.denominator})

print("\(fracSortedByDenominator[0].name) < \(fracSortedByDenominator[1].name) < \(fracSortedByDenominator[2].name) because \(fracSortedByDenominator[0].denominator) > \(fracSortedByDenominator[1].denominator) > \(fracSortedByDenominator[2].denominator)")
```

Figure 36

Example 6



Note. Student work sample 6.

Description: Common denominator, benchmark numbers, includes list of multiples to aide in finding the LCM, shows one set of common denominator calculations to compare $\frac{2}{3}$ and $\frac{3}{4}$, benchmark number of $\frac{1}{2}$ to determine the relative position of $\frac{3}{8}$

New inputs: It makes sense to keep the new inputs small whole numbers. Try $\frac{1}{3}, \frac{3}{5}, \frac{3}{8}$

Given inputs: $\frac{2}{3}, \frac{3}{4}, \frac{3}{8}$ the machine produces the following:

2/3 is greater than 1/2

3/4 is greater than 1/2

3/8 is less than 1/2

2/3 -> 8/12

3/4 -> 9/12

3/8 < 2/3 < 3/4

Given inputs $\frac{1}{3}, \frac{3}{5}, \frac{3}{8}$, the machine produces:

1/3 is less than 1/2

3/5 is greater than 1/2

3/8 is less than 1/2

1/3 -> 8/24

3/8 -> 9/24

1/3 < 3/8 < 3/5

Code used to produce those two outputs (toggle initial group of “let frac1...” and untoggle the next group to produce the second output):

```
import UIKit

class Fractions {
    let name: String
    var numerator: Int
    var denominator: Int
    init(name: String, numerator: Int, denominator: Int) {
        self.name = name
        self.numerator = numerator
        self.denominator = denominator
    }
}

let frac1 = Fractions(name: "2/3", numerator: 2, denominator: 3)
let frac2 = Fractions(name: "3/4", numerator: 3, denominator: 4)
let frac3 = Fractions(name: "3/8", numerator: 3, denominator: 8)

//let frac1 = Fractions(name: "1/3", numerator: 1, denominator: 3)
//let frac2 = Fractions(name: "3/5", numerator: 3, denominator: 5)
//let frac3 = Fractions(name: "3/8", numerator: 3, denominator: 8)


var frac1A = frac1
```

```
var frac2A = frac2
var frac3A = frac3




var fractions: [Fractions] = [frac1, frac2, frac3]


var lessThanHalf: [Fractions]
var greaterThanHalf: [Fractions]
lessThanHalf = []
greaterThanHalf = []


for fracs in fractions {
    if Double(fracs.numerator)/Double(fracs.denominator) <= 0.5 {
        lessThanHalf.append(fracs)
        print("\(fracs.name) is less than 1/2")
    } else {
        if Double(fracs.numerator)/Double(fracs.denominator) >= 0.5 {
            greaterThanHalf.append(fracs)
            print("\(fracs.name) is greater than 1/2")
        }
    }
}
```

```
var modAB: Int
func gcd(a: inout Fractions, b: inout Fractions) {
    var i: Int = 1
    var Amultiplier: Int = 0
    var Bmultiplier: Int = 0
    var LCMA: Int = 0
    var modAB: Int = 1
    while modAB != 0 {
        i = i + 1
        LCMA = a.denominator * i
        modAB = LCMA % b.denominator
        Amultiplier = LCMA / a.denominator
        Bmultiplier = LCMA / b.denominator
    }
    a.denominator = a.denominator * Amultiplier
    a.numerator = a.numerator * Amultiplier
    b.denominator = b.denominator * Bmultiplier
    b.numerator = b.numerator * Bmultiplier
    //    return (a, b)
}
```

```
//var newA = gcd(a: &frac1A, b: &frac2A)
//print("\(frac1.name) -> \(frac1A.numerator)/\(frac1A.denominator)")


//if the count of the less than halves is bigger than 1, then a GCD algorithm makes sense. Otherw
ise, the count of the ones greater than a half ought to be bigger than 1, so the GCD algorithm sho
uld run over the others.

var countGreater = greaterThanHalf.count
var countLess = lessThanHalf.count
var fractionsNew: [Fractions]


if countGreater == 2 {
    var greaterThanHalfA: Fractions = greaterThanHalf[0]
    var greaterThanHalfB: Fractions = greaterThanHalf[1]
    gcd(a: &greaterThanHalfA, b: &greaterThanHalfB)
    print("\(greaterThanHalfA.name) -> \(greaterThanHalfA.numerator)/\(greaterThanHalfA.deno
minator)")
    print("\(greaterThanHalfB.name) -> \(greaterThanHalfB.numerator)/\(greaterThanHalfB.deno
minator)")
    fractionsNew = [greaterThanHalfA, greaterThanHalfB]
```

```
    let fracSortedByNumerator = fractionsNew.sorted(by: { $0.numerator < $1.numerator})
    print("\(lessThanHalf[0].name) < \(fracSortedByNumerator[0].name) < \(fracSortedByNumera
tor[1].name)")

} else {
    if countLess == 2 {
        var lessThanHalfA = lessThanHalf[0]
        var lessThanHalfB = lessThanHalf[1]
        gcd(a: &lessThanHalfA, b: &lessThanHalfB)
        print("\(lessThanHalfA.name) -> \(lessThanHalfA.numerator)/\(lessThanHalfA.denominato
r)")
        print("\(lessThanHalfB.name) -> \(lessThanHalfB.numerator)/\(lessThanHalfB.denominator
)")
        fractionsNew = [lessThanHalfA, lessThanHalfB]
        let fracSortedByNumerator = fractionsNew.sorted(by: { $0.numerator < $1.numerator})
        print("\(fracSortedByNumerator[0].name) < \(fracSortedByNumerator[1].name) < \(greater
ThanHalf[0].name)")

    } else {
        print("IDK")
    }
}
//print("\(newA.name) -> \(newA.numerator)/\(newA.denominator)")
```

```
//let fracSortedByDenominator = fractions.sorted(by: { $0.denominator < $1.denominator})
```

Discussion of student work examples

While the process for establishing projective validity is admittedly cumbersome to produce and onerous to read about, the notion that inferences that habitually take place 'under the hood' in mathematics education – inferring what a student would do in some new circumstance - might have three breaking points instead of just one implies that the 'science' this process produces is stronger than those everyday kinds of inferences. My point in including the rather crude code is not so much to say "this is the way it ought to be done" and more to say "this is one way to make everyday inferences falsifiable." I expect readers to strongly disagree that the machine coding is a good model of student thinking. It is not. It is unsophisticated compared to the kinds of models that abound in mathematics education. While more sophisticated models certainly exist, the care needed to mechanize those models to formalize their predictions is, I take it, under explored. The mathematical point for doing this work would be to test to see how far algorithmic elaboration can get us within a particular domain. I suspect that basic arithmetic can be fully algorithmically elaborated but also suspect that calculus might involve dialectic, rather than algebraic thinking. This is part of what Marx explores in asking questions like "The entire difficulty in understanding the differential operation (as in that of any *negation of negation* whatever) lies precisely in seeing *how* it differs from such a simple procedure" (Gumbel & Yanovskaya, 2018, p. 1).

Conclusion

In this chapter, I began by discussing how meaning fields relate to compossible regions in the manifold. This allowed me to express the error in hybridized models descriptively. I then used the existence of hybridized models as justification for concluding that numbers are anaphoric terms. While hybridized models are statistically insignificant, my suspicion is that the kind of thinking involved with those models is more commonplace than it might appear. I suspect that as students develop representational vocabularies, the inability to treat disjunction in the exclusive sense results in the state of confusion which reduces the salience of all spatial representations of quantity. This would explain why students seem to drop the common representations - that are intended to clarify rather than confuse - even if they are still making errors that those models might conceptually help with.

From that discussion I moved on to the Hermeneutic Calculator. Developing the code for this calculator involves projection from student work samples to machine-produced examples. To put some boundaries on how this can happen, I introduced the notion of projective validity. I think projective validity has relevance outside of the mechanical purposes I put it to. Projective inferences are falsifiable at each of the three constituent inferences. It seems that this feature allows for the everyday kinds of inferences about student thinking that mathematics educators rely on could be made more formal by analyzing them as I have demonstrated.